

Arquitectura Local para Generación Fotorrealista: Entrenamiento LoRA (SDXL) y Despliegue Nodal Avanzado

Autor: DanielCostas

Hardware de Despliegue: NVIDIA GeForce RTX 3060 (12GB VRAM)

Descripción del Proyecto

Este repositorio documenta el diseño integral, la ingeniería de datos, la optimización de entrenamiento y el despliegue de un entorno local de Inteligencia Artificial para la generación de imágenes fotorrealistas de alta fidelidad. El objetivo principal es consolidar un *pipeline* estable y altamente configurable, superando las limitaciones técnicas de las interfaces lineales tradicionales mediante la adopción de una arquitectura basada en grafos (ComfyUI).

Aviso de Seguridad Operacional (OPSEC): Cumpliendo con los estándares de privacidad y protección de datos biométricos, el *dataset* original de imágenes, los binarios de los modelos base (archivos .safetensors de >6GB) y los pesos de la red neuronal entrenada no se incluyen en este repositorio. Se emplea un archivo .gitignore estricto para bloquear la subida de datos sensibles, cachés locales y directorios de trabajo. Este repositorio expone exclusivamente la lógica de automatización, la arquitectura de nodos y las configuraciones de los hiperparámetros.

Fase 1: Ingeniería de Datos y Pipeline de Preprocesamiento

La calidad del modelo resultante depende de la preparación inicial del conjunto de datos, compuesto por 31 fotografías. Se ha desarrollado un *pipeline* secuencial mediante cinco scripts en Python para garantizar la máxima pureza de los datos antes de su ingesta en Kohya_ss:

1. preprocess.py (**Estandarización Geométrica**): Detecta el modo de color de las imágenes crudas, calcula un recorte central perfecto (*center-crop*) y aplica un algoritmo de remuestreo LANCZOS para redimensionar la salida exactamente a 1024x1024 píxeles, la resolución nativa óptima para SDXL.

2. `rename_dataset.py` (**Trazabilidad**): Sanea los nombres de archivo originales, aplicando un renombrado masivo y secuencial (ej. `img_001.jpg` a `img_031.jpg`) tanto en el directorio de origen como en el procesado.
3. `generate_captions.py` (**Base Semántica**): Script de inicialización que genera archivos de texto plano independientes y emparejados por cada imagen, inyectando el *trigger word* principal de activación: `ohwx man`.
4. `auto_tagger.py` (**Etiquetado Vision-Language**): Implementación del modelo BLIP de Salesforce. Se carga en la memoria VRAM para analizar visualmente el *dataset* y generar un etiquetado extenso del entorno, iluminación y vestuario.
5. `clean_taggers.py` (**Purga y Post-procesamiento**): Script crítico para optimizar el aprendizaje. Sobrescribe los archivos `.txt` generados en el paso anterior para purgar descripciones excesivas, aislando el concepto base. Esto fuerza a la red neuronal a vincular los rasgos biométricos de manera exclusiva al identificador principal, evitando ruido semántico.

Fase 2: Optimización del Entrenamiento (Kohya_ss)

El ajuste fino (*Fine-Tuning*) de la identidad se realizó entrenando un modelo LoRA (*Low-Rank Adaptation*) sobre la arquitectura SDXL. Se corrigieron y auditaron los parámetros de configuración en formato JSON para alinear el entrenamiento con las capacidades del hardware local.

- **Resolución de Conflictos de Arquitectura:** Se configuró explícitamente el parámetro `"model_type": "sdxl"`, corrigiendo desajustes previos con topologías incompatibles (FLUX).
- **Precisión Numérica Unificada:** Optimizado específicamente para la arquitectura Ampere. Se establecieron los parámetros `mixed_precision` y `save_precision` en `bf16` (*Bfloat16*), maximizando el rendimiento de la GPU y evitando errores de tensores.
- **Anonimización de Entorno:** Se sustituyeron todas las rutas absolutas del sistema de archivos local por rutas relativas genéricas (ej. `./dataset/img`), garantizando la portabilidad y seguridad de los archivos de configuración expuestos.

Fase 3: Infraestructura y Enrutamiento (ComfyUI)

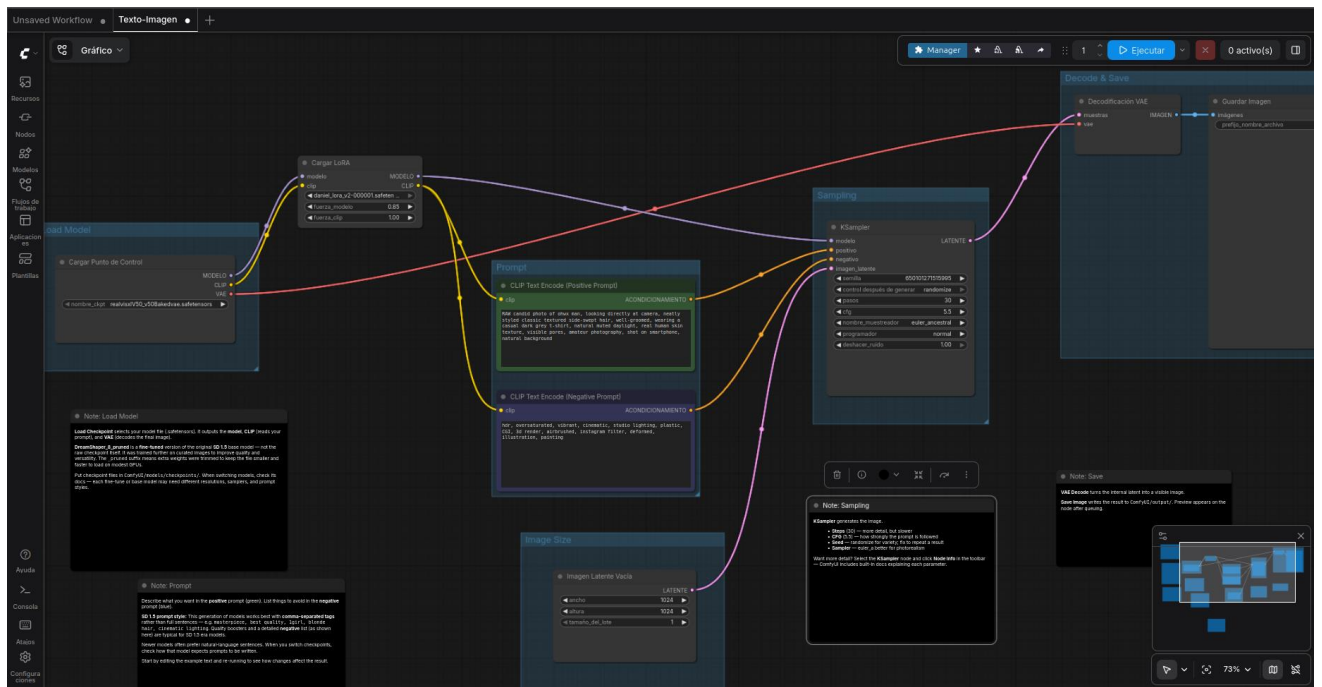
Para habilitar técnicas de fotorrealismo de grado forense, se descartó el uso de interfaces lineales (Forge) en favor de **ComfyUI**. La infraestructura se preparó priorizando la eficiencia del almacenamiento y la gestión de dependencias.

- **Aislamiento de Entorno:** Despliegue sobre un entorno virtual Python (venv) independiente, equipado con los tensores base de PyTorch y soporte nativo CUDA 12.1.
- **Enrutamiento de Almacenamiento Compartido:** Configuración del archivo `extra_model_paths.yaml` mediante rutas absolutas. Esto permite a ComfyUI leer Checkpoints (RealVisXL V5.0 de 6.5 GB), VAEs y LoRAs directamente desde el directorio de instalación original de Forge, eliminando la duplicación masiva de datos en el disco.
- **Gestor de Dependencias:** Instalación de ComfyUI-Manager en el directorio de nodos personalizados para centralizar y automatizar el despliegue de extensiones de terceros.

Fase 4: Arquitectura del Grafo de Inferencia (El Flujo Nodal)

El diagrama de flujo principal se diseñó interceptando la señal del modelo base para aplicar los pesos de identidad antes de la decodificación latente, logrando una representación anatómica coherente.

- **Intercepción LoRA:** La señal proveniente de Load Checkpoint (RealVisXL) se redirige a través de un nodo Load LoRA (Fuerza: 0.85).
- **Bifurcación Semántica (CLIP):** El condicionamiento de texto pasa a través del LoRA hacia dos nodos de codificación separados. El *prompt* positivo refuerza la fidelidad estética, incluyendo directrices específicas sobre el peinado (*classic textured side-swept hair*) y texturas orgánicas (*real human skin texture, visible pores*).
- **Motor de Muestreo (KSampler):** Parametrizado para maximizar el detalle sin sobre-quemar la imagen.



Parámetro KSampler	Valor Configurado
Resolución Latente	1024 x 1024
Pasos (Steps)	30
Escala CFG	5.5
Muestreador (Sampler)	euler_ancestral
Programador (Scheduler)	normal

Fase 5: Hoja de Ruta y Escalabilidad Modular

El objetivo del sistema es erradicar el "valle inquietante" y el sangrado de conceptos (*Concept Bleeding*), común cuando la IA intenta generar tatuajes o micro-detalles en

planos generales. El desarrollo próximo implementará una **Arquitectura de 3 LoRAs Concurrentes**, basándose en enmascaramiento espacial:

Capa 1: LoRA Global (Identidad y Complejión)

- **Dataset:** 50% planos medios, 20% planos generales.
- **Implementación:** Al inicio del flujo nodal. Establece la volumetría corporal, postura y encuadre general del personaje.

Capa 2: LoRA Regional (Arte Corporal / Tatuajes)

- **Dataset:** Mapeo fotográfico de 360 grados enfocado exclusivamente en las extremidades.
- **Implementación:** Integración mediante nodos de *Attention Masking* (Regional Prompting) o proyecciones vía IPAdapter. Se restringirá la inyección de los pesos neuronales de la tinta únicamente a las áreas detectadas correspondientes a los brazos, protegiendo el resto de la composición.

Capa 3: LoRA Facial (Micro-Restauración - FaceDetailer)

- **Dataset:** 30% de la carga total del conjunto de datos compuesto por primeros planos extremos, bajo iluminación plana.
- **Implementación:** Integrado al final del flujo principal. Se utilizará un nodo detector para hacer un recorte automático del rostro de la imagen base. Sobre ese fragmento, se aplicará el tercer LoRA a alta resolución seguido de un proceso de *Inpainting* dinámico, fusionando la textura perfecta de la piel y los reflejos oculares con la imagen original.